WebScrapper Empresas JM

Documentación técnica y manual de uso

Captación de empresas locales (Empresite / Páginas Amarillas), gestión en MySQL y envío de campañas de email con control de warm-up

Repositorio	danmelendo/webscrapperEmpresasJM
Versiones documentadas	WebScrapper_DAGM_ver1_empresite.py · WebScrapper_DAGM_ver6.py · Consultor_db_v5.py
Commit base	211e82e (semana 2 de prácticas, 19/02/2026)
Fecha del documento	30/09/2026

Índice

Índice 2

1. Visión general 3

1.1 Flujo de datos . . . 3

1.2 Contenido del repositorio . . . 3

2. Instalación 4

2.1 Requisitos . . . 4

2.2 Pasos . . . 4

2.3 Configuración (.env) . . . 4

3. Scraper de Empresite (Selenium) 5

3.1 Campos de la interfaz . . . 5

3.2 Funcionamiento interno . . . 5

3.3 Ficheros de salida . . . 6

3.4 Parámetros ajustables en el código . . . 6

4. Scraper de Páginas Amarillas (requests) 7

5. Base de datos 8

5.1 Catálogos . . . 8

6. Consultor de base de datos y envío de emails 10

6.1 Pestaña «Empresas con Email» . . . 10

6.2 Ventana «Enviar email» . . . 10

6.3 Pestañas «Empresas con Teléfono» y «Estado Emails» . . . 12

6.4 Pestaña «Mantenimiento» . . . 12

7. Pruebas realizadas 14

8. Problemas conocidos y recomendaciones 15

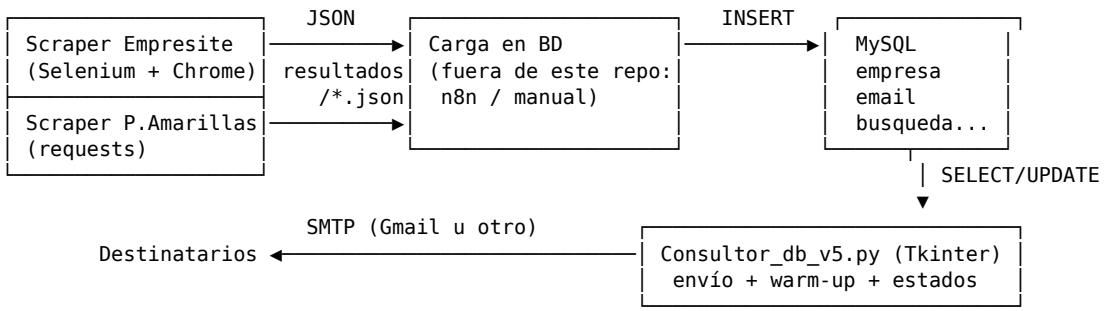
8.1 Uso responsable . . . 15

1. Visión general

El proyecto automatiza la **captación comercial de empresas locales** para JMOrdenadores. Está formado por dos piezas independientes que se comunican a través de una base de datos MySQL:

- **Scrapers** (Python + BeautifulSoup/Selenium): recorren directorios de empresas y generan ficheros JSON con nombre, teléfono, email, web, dirección y emails «posibles» deducidos del dominio.
- **Consultor de base de datos** (Consultor_db_v5.py): aplicación de escritorio Tkinter que lee las empresas de MySQL, permite enviar campañas de email HTML personalizadas vía SMTP con límites de envío progresivos (warm-up), registra el estado de cada email y ofrece herramientas de limpieza de datos.

1.1 Flujo de datos



Nota. El paso «JSON → base de datos» **no está implementado en este repositorio**. Una versión antigua (WebScrapper_DAGM_ver3.py) enviaba los datos a un webhook de n8n; la carga actual se hace fuera del código versionado. En la sección 5 se describe el esquema que el consultor espera encontrar.

1.2 Contenido del repositorio

Archivo	Estado	Descripción
WebScrapper_DAGM_ver1_empresite.py	**Actual**	Scraper de empresite.eleconomista.es con Selenium/Chrome. GUI Tkinter, checkpoints por página, deduplicación.
WebScrapper_DAGM_ver6.py	**Actual**	Scraper de paginasamarillas.es con requests. GUI Tkinter. Un JSON por página.
Consultor_db_v5.py	**Actual**	Consultor MySQL + envío de emails + mantenimiento de datos.
WebScrapper_DAGM_ver1/2/3/5.py	Histórico	Iteraciones previas del scraper de Páginas Amarillas (v3 incluye webhook n8n).
consultor_db_v3.py	Histórico	Versión anterior del consultor (texto plano, sin warm-up).
warmup_state.json	Estado	Contador de envíos por día y marcas de tiempo (límite horario).
recontact_state.json	Estado	Último mes en que se reactivaron los emails enviados.
selenium_profile_empresite/	Generado	Perfil de Chrome usado por Selenium. No debería versionarse (ver 8).
docs/	Nuevo	Esta documentación, capturas y esquema SQL de referencia.
requirements.txt, .env.example	Nuevo	Dependencias y plantilla de configuración.

2. Instalación

2.1 Requisitos

Componente	Versión / notas
Python	3.10 o superior (probado con 3.12). En Windows, el instalador oficial ya incluye Tkinter.
Google Chrome	Necesario solo para el scraper de Empresite. Selenium ≥ 4.6 descarga el ChromeDriver adecuado automáticamente (Selenium Manager).
MySQL / MariaDB	Necesario para el consultor. Probado con MariaDB 10.11.
Cuenta SMTP	Gmail con verificación en dos pasos + **contraseña de aplicación**, u otro servidor SMTP.

2.2 Pasos

```
git clone https://github.com/danmelendo/webscrapperEmpresasJM.git
cd webscrapperEmpresasJM
python -m venv .venv
.venv\Scripts\activate          # Windows   (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
copy .env.example .env           # Windows   (Linux/macOS: cp .env.example .env)
# editar .env con las credenciales reales
```

2.3 Configuración (.env)

Solo el consultor usa .env. Se busca en la **misma carpeta que el script** (o que el ejecutable si se empaqueta con PyInstaller), no en el directorio de trabajo.

Variable	Por defecto	Uso
DB_HOST / DB_PORT	localhost / 3306	Servidor MySQL.
DB_USER / DB_PASSWORD	—	Credenciales de la BD.
DB_NAME	—	Base de datos. Obligatoria: se usa también para inspeccionar information_schema.
DB_CHARSET	utf8mb4	Juego de caracteres de la conexión.
SMTP_SERVER / SMTP_PORT	smtp.gmail.com / 587	587 → STARTTLS; 465 → SSL directo.
SMTP_USER / SMTP_PASS	—	Remitente y contraseña. Sin ellas el envío falla con un error explícito.

3. Scraper de Empresite (Selenium)

Archivo: WebScrapper_DAGM_ver1_empresite.py · **Ejecución:** `python WebScrapper_DAGM_ver1_empresite.py`

Abre un Chrome visible controlado por Selenium, recorre las páginas del listado de empresite.eleconomista.es, entra en la ficha de cada empresa y extrae email, web y teléfono.

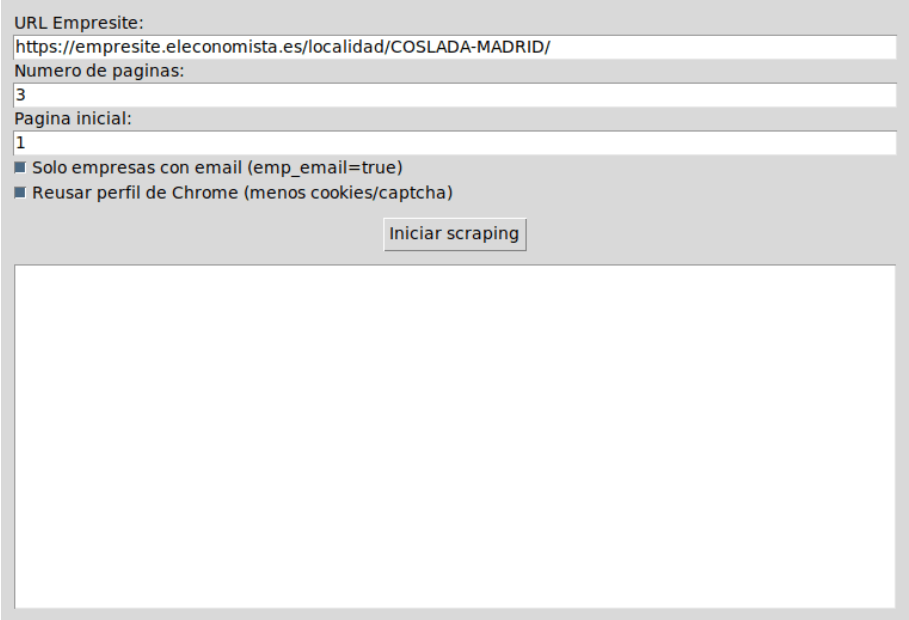


Figura 1 – Ventana del scraper de Empresite

3.1 Campos de la interfaz

Campo	Descripción
URL Empresite	URL de un listado de empresite (p. ej. `https://empresite.eleconomista.es/localidad/COSLADA-MADRID/`). Copiarla del navegador tras hacer la búsqueda. Debe ser del dominio empresite.eleconomista.es o el scraper se detiene.
Número de páginas	Página **final** (inclusive) hasta la que se recorre.
Página inicial	Página desde la que empezar. Útil para **reanudar** un scraping interrumpido.
Solo empresas con email	Añade `?testfiltros=1&emp_email=true` a la URL (filtro propio de Empresite).
Reusar perfil de Chrome	Usa la carpeta selenium_profile_empresite/ para conservar cookies y reducir banners/captchas.

3.2 Funcionamiento interno

- Paginación: la página N se construye añadiendo `/PgNum-N/` a la ruta (la página 1 no lleva sufijo).
- En cada listado se recogen los enlaces a fichas (`/NOMBRE-EMPRESA.html` en la raíz o rutas `/empresa/`), incluidos los que usan `onclick="location.href=…"`. Se descartan páginas legales/soporte (privacidad, cookies, términos…).
- Nombre: texto del enlace; si es «Ver ficha» se deduce de la URL (TALLERES-PEREZ-SL.html → «Talleres Perez Sl»).
- En la ficha: email de `a.email[href^=mailto:]`, web de `a.url`, teléfono de `tel:` o por regex (9 dígitos / +34), normalizado a +34XXXXXXXXX.
- Con el dominio de la web (o del email) se generan **info@**, **contacto@** y **administracion@**.
- Si la ficha no muestra email en 3 s, se hace una **pausa de 7 s** para que el usuario pueda resolver un captcha o banner en la ventana de Chrome.

- Deduplicación global por URL de ficha: una empresa que aparece en varias páginas solo se procesa una vez.

3.3 Ficheros de salida

Se escriben en resultados/ (relativo al directorio desde el que se lanza el script). El nombre deriva de la ruta de la URL:

Fichero	Contenido
localidad_COSLADAMADRID.json	Acumulado. Se reescribe tras cada página (campo `pagina_hasta`) y al final.
localidad_COSLADAMADRID_pg001.json …	Checkpoint con solo las empresas nuevas de esa página.


```
{
    "localidad": "Coslada Madrid",
    "tipo_empresa": "localidad",
    "pagina_hasta": 2,
    "resultados": [
        {
            "nombre": "TALLERES PEREZ SL",
            "telefono": "+34916123456",
            "email": "info@talleresperez.es",
            "email_posible_info": "info@talleresperez.es",
            "email_posible_contacto": "contacto@talleresperez.es",
            "email_posible_administracion": "administracion@talleresperez.es",
            "web": "https://www.talleresperez.es",
            "direccion": "No disponible",
            "codigo_postal": "No disponible",
            "localidad": "Coslada Madrid",
            "url_detalle": "https://empresite.eleconomista.es/TALLERES-PEREZ-SL.html"
        }
    ]
}
```


Los campos sin dato valen la cadena `"No disponible"`. Empresite no rellena dirección ni código postal en esta versión.

3.4 Parámetros ajustables en el código

Constante	Valor	Efecto
CAPTCHA_CHECK_ENABLED	False	Detección automática de captcha desactivada (daba falsos positivos).
BROWSER_HIDE_ENABLED	False	Si True, mueve Chrome fuera de pantalla (Windows).
DETAIL_DELAY_SECONDS / PAGE_DELAY_SECONDS	(0, 0)	Retardos entre fichas / páginas (actualmente sin uso: máxima velocidad).
EXCLUDE_HTML_PAGES / _PATTERNS	lista	Páginas que no son fichas de empresa.

Nota. Rendimiento medido: la búsqueda de banners de cookies prueba 9 textos con 3 s de espera cada uno, así que **cada carga de página sin banner tarda ~27 s extra**, más 7 s por ficha sin email. En la prueba, 2 páginas y 3 fichas tardaron 139 s. Para listados grandes, reducir el timeout de intentar_aceptar_cookies o llamarlo solo en la primera carga.

4. Scraper de Páginas Amarillas (requests)

Archivo: WebScrapper_DAGM_ver6.py · **Ejecución:** `python WebScrapper_DAGM_ver6.py`

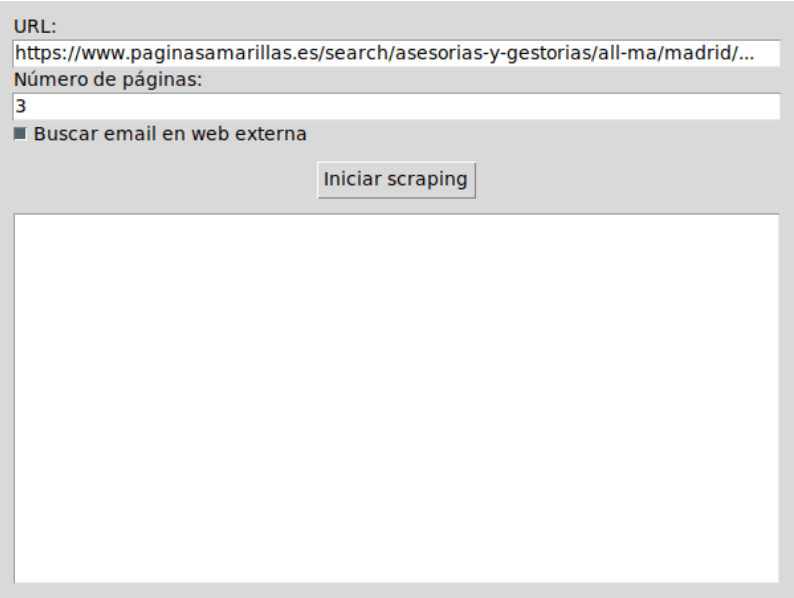


Figura 2 – Ventana del scraper de Páginas Amarillas

- **URL**: URL de resultados de paginasamarillas.es con parámetros `what` (actividad) y `where` (localidad).
- **Número de páginas**: cuántas páginas recorrer (por defecto 3).
- **Buscar email en web externa**: si la ficha no trae email, descarga la web de la empresa y busca un email **del mismo dominio**.
- Extrae nombre, teléfono, email, web, dirección, código postal y localidad (microdatos `itemprop`) de cada bloque `div.box`.
- Salida: resultados/<What><Where>_pagina_N.json (p. ej. AsesoriasygestoriasCoslada_pagina_1.json), con `localidad`, `tipo_empresa` y `resultados`. Pausa aleatoria de 0,3–0,7 s entre empresas.

Atención. Error conocido: construir_url() busca el patrón `/all-nc/"1"` (con comillas), que no aparece en las URLs reales. Resultado: **todas las «páginas» descargan la página de la URL original** y se generan N ficheros idénticos. Corrección sugerida: `re.sub(r'(/all-nc/)\d+', rf'\g<1>{pagina}', base_url)`.

5. Base de datos

El consultor no crea las tablas principales; espera un esquema existente. El siguiente esquema se ha **deducido de las consultas del código** y es con el que se han realizado las pruebas. Está disponible en docs/schema_referencia.sql.

```
CREATE DATABASE IF NOT EXISTS empresas_jm CHARACTER SET utf8mb4;
USE empresas_jm;

CREATE TABLE empresa (
  id_empresa    INT AUTO_INCREMENT PRIMARY KEY,
  nombre        VARCHAR(255),
  telefono      VARCHAR(20),
  web           VARCHAR(255),
  direccion     VARCHAR(255),
  codigo_postal VARCHAR(10),
  localidad     VARCHAR(120)
);

-- Catálogo: RE (real), IN (info@), CO (contacto@), AD (administracion@)
CREATE TABLE tipo_email (
  id_tipo_email CHAR(2) PRIMARY KEY,
  descripcion   VARCHAR(50)
);

-- Catálogo: PE (pendiente), EN (enviado), ER (error)
CREATE TABLE estado_email (
  id_estado   CHAR(2) PRIMARY KEY,
  descripcion VARCHAR(50)
);

CREATE TABLE email (
  id_email      INT AUTO_INCREMENT PRIMARY KEY,
  id_empresa    INT NOT NULL,
  id_tipo_email CHAR(2),
  email         VARCHAR(255),
  id_estado     CHAR(2) NULL,
  FOREIGN KEY (id_empresa)    REFERENCES empresa(id_empresa),
  FOREIGN KEY (id_tipo_email) REFERENCES tipo_email(id_tipo_email),
  FOREIGN KEY (id_estado)     REFERENCES estado_email(id_estado)
);

-- Cada ejecución del scraper (actividad + localidad)
CREATE TABLE busqueda (
  id_busqueda  INT AUTO_INCREMENT PRIMARY KEY,
  tipo_empresa VARCHAR(120),
  localidad    VARCHAR(120),
  fecha        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE busqueda_empresa (
  id_busqueda INT NOT NULL,
  id_empresa  INT NOT NULL,
  PRIMARY KEY (id_busqueda, id_empresa),
  FOREIGN KEY (id_busqueda) REFERENCES busqueda(id_busqueda),
  FOREIGN KEY (id_empresa)  REFERENCES empresa(id_empresa)
);

INSERT INTO tipo_email VALUES
  ('RE','Real'), ('IN','Posible info@'), ('CO','Posible contacto@'), ('AD','Posible administracion@');
INSERT INTO estado_email VALUES
  ('PE','Pendiente'), ('EN','Enviado'), ('ER','Error');
```

5.1 Catálogos

Tabla	Código	Significado
tipo_email	RE	Email **real** (obtenido de la ficha o de la web). Corresponde al campo `email` del JSON.
	IN / CO / AD	Emails **posibles** info@ / contacto@ / administracion@ (campos `email_posible_*`).

Tabla	Código	Significado
estado_email	PE	Pendiente: se marca justo antes de intentar el envío.
	EN	Enviado correctamente.
	ER	Error en el envío.

Esquemas alternativos soportados. Para el estado del email, el consultor inspecciona information_schema y admite, por orden: (1) una columna de estado en email (`id_estado`, `id_estado_email`, `estado_email` o `estado`); (2) estado_email con referencia `id_email` o `email`; (3) si estado_email es un catálogo con `descripcion`, crea automáticamente la tabla email_estado(id_email, id_estado, fecha_actualizacion).

Tipo de empresa y localidad usados en las plantillas salen de la busqueda más reciente (`MAX(id_busqueda)`) asociada a la empresa en busqueda_empresa. Si faltan, se usan «empresa» y «Madrid».

6. Consultor de base de datos y envío de emails

Archivo: Consultor_db_v5.py · **Ejecución:** `python Consultor_db_v5.py` (con .env junto al script).

Al arrancar ejecuta el **recontacto mensual**: si el mes actual es distinto al guardado en recontact_state.json, todos los emails en estado EN pasan a PE para poder volver a contactarlos ese mes.

6.1 Pestaña «Empresas con Email»

Pulsar **Cargar empresas con email**. Se cargan todos los emails válidos (excluye «no disponible», vacío, none, null) y se clasifican en: reales (RE), posibles (IN/CO/AD) y ya enviados (estado EN). La lista principal muestra solo los reales.

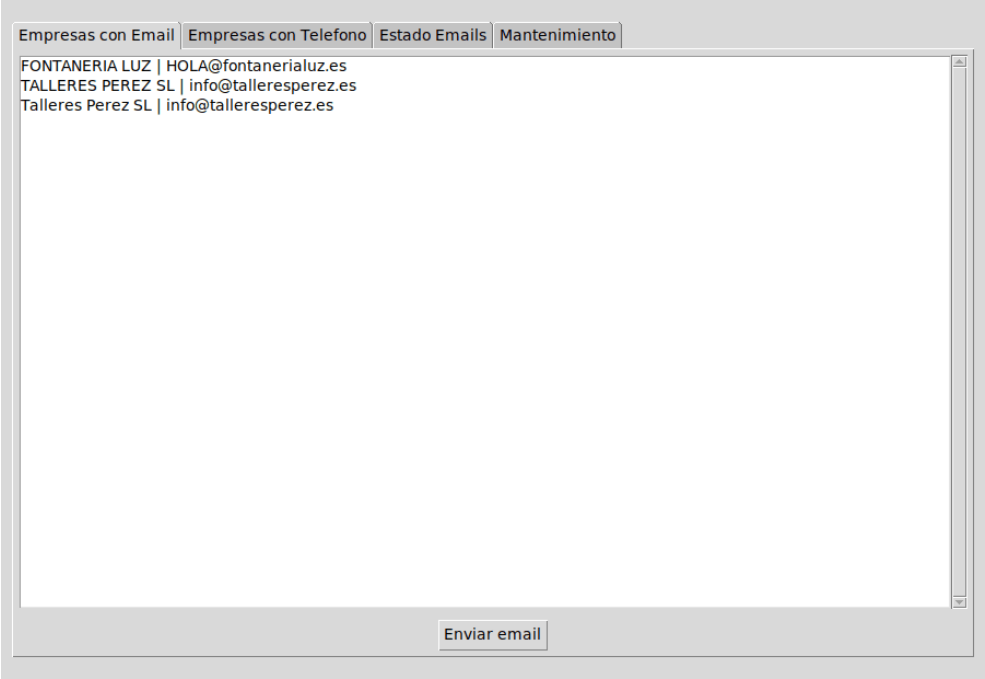


Figura 3 – Consultor: emails reales cargados

6.2 Ventana «Enviar email»

Botón **Enviar email** (requiere haber cargado antes). Contiene tres pestañas de destinatarios: **Reales (RE)**, **Posibles (IN/CO/AD)** y **Emails ya enviados** (solo consulta). Se pueden seleccionar varios con Ctrl/Mayús o con **Seleccionar todas**.

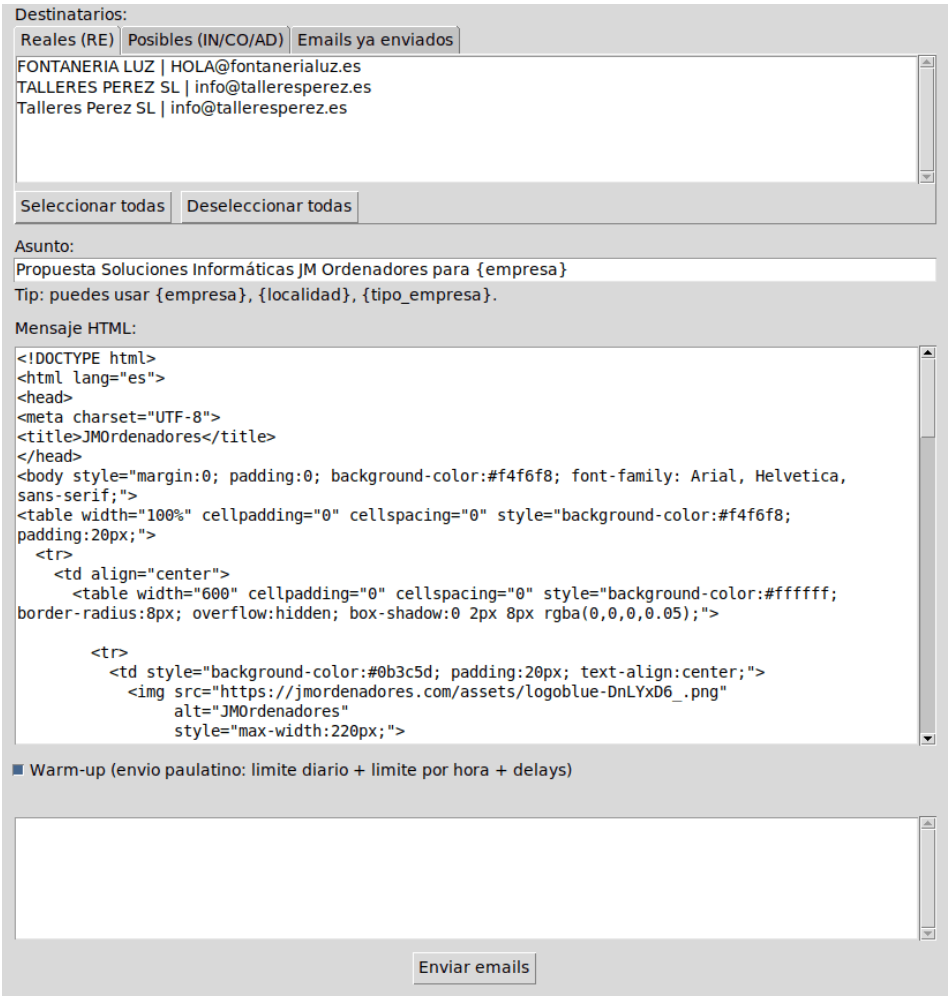


Figura 4 – Ventana de envío con plantilla HTML y opción de warm-up

Plantillas de asunto y cuerpo

Marcador	Asunto	Cuerpo	Valor
{empresa}	Sí	Sí	Nombre de la empresa (o «su empresa» / «vuestra empresa»).
{tipo_empresa}	Sí	Sí	Actividad de la búsqueda (o «empresa»).
{localidad}	Sí	Sí	Localidad de la búsqueda (o «Madrid»).
{saludo}	No	Sí	«Estimado equipo de <empresa>,» o «Estimado equipo,».

Importante. El cuerpo se procesa con `str.format`: **cualquier llave literal** (por ejemplo CSS en un bloque `<style>p{color:red}</style>`) hace fallar el envío de todos los emails con error `'color'` y los marca ER. Usar estilos en línea (`style="…"`) o escribir las llaves dobles `{{ }}`. En el asunto, un marcador desconocido hace que **no se sustituya ninguno**.

Proceso de envío

- Se eliminan destinatarios repetidos (email normalizado en minúsculas) de la selección.
- Por cada email: estado PE → envío SMTP → estado EN (o ER si falla). Los errores se listan al final en un diálogo.
- El envío se ejecuta en un hilo; el registro inferior muestra ENVIADO / ERROR / pausas en tiempo real.

Warm-up (activado por defecto)

Protege la reputación del remitente limitando el ritmo. El estado se guarda en warmup_state.json y el día 0 es la fecha del primer envío registrado.

Día desde inicio	0	1	2	3	4	5	6	7	8+
Límite diario	10	20	30	40	60	80	100	120	150

- Máximo **15 emails por hora** (ventana móvil).
- Espera aleatoria de **25–75 s** entre emails y pausa larga de **2–5 min** cada 5 envíos.
- Al alcanzar un límite, el envío **se detiene** (no espera): hay que volver a lanzarlo más tarde. Los ya enviados no reaparecen en «Reales» porque quedan en estado EN.

6.3 Pestañas «Empresas con Teléfono» y «Estado Emails»

Listados de consulta: empresas con teléfono válido (Cargar telefonos) y el estado de cada email con su descripción (Consultar estados de email).

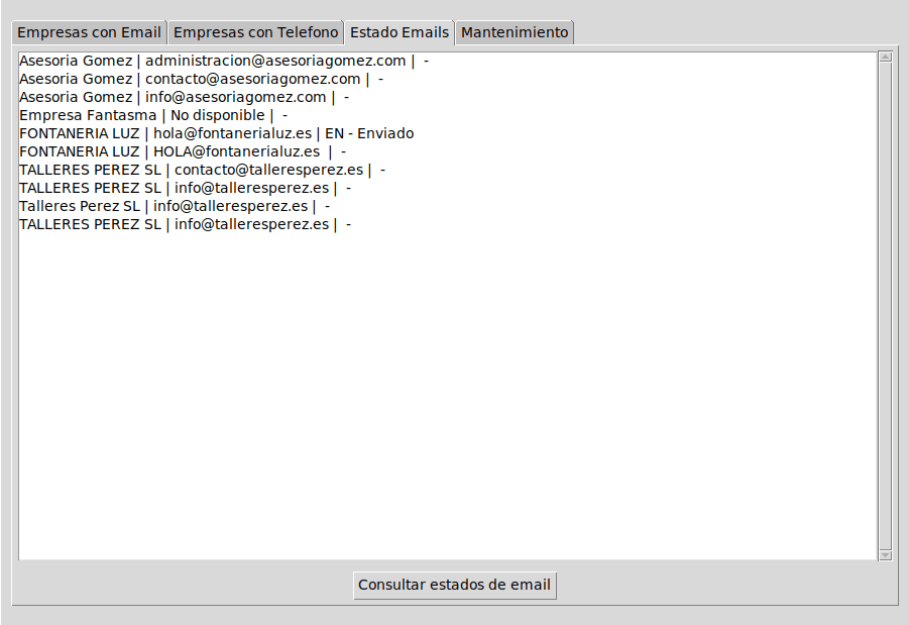


Figura 5 – Consulta de estados de email

6.4 Pestaña «Mantenimiento»

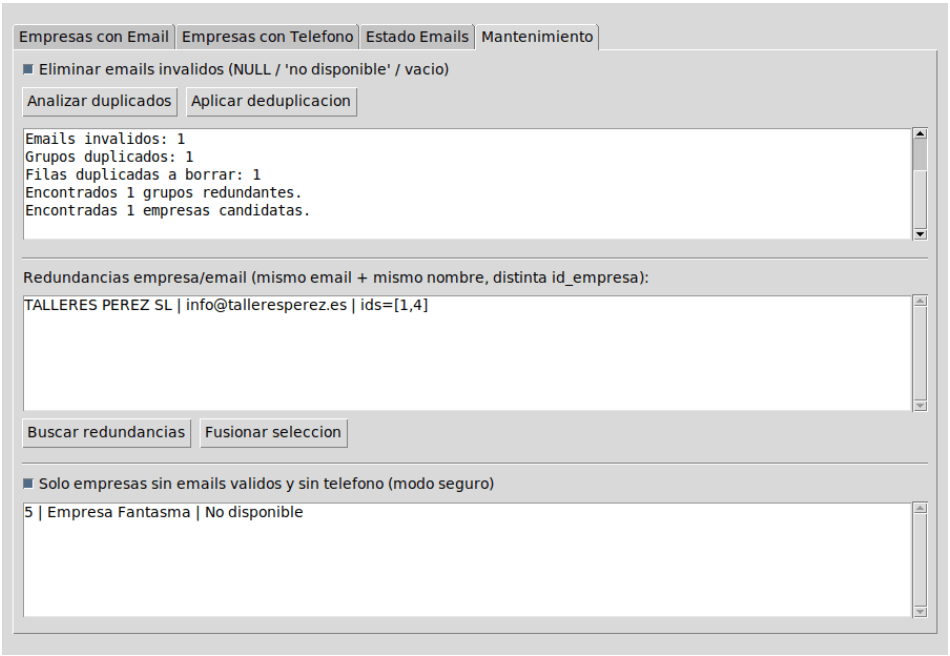


Figura 6 – Herramientas de mantenimiento

Acción	Qué hace
Analizar duplicados	Cuenta emails inválidos y filas duplicadas (mismo email normalizado + misma empresa + mismo tipo). No modifica nada.
Aplicar deduplicación	Borra los duplicados conservando el id más bajo. Con la casilla marcada, borra también los emails inválidos. Pide confirmación.
Buscar redundancias / Fusionar selección	Detecta empresas distintas con el mismo nombre y el mismo email (empresas duplicadas). Fusionar mueve emails, búsquedas y estados a la empresa de id más bajo, completa sus campos vacíos (teléfono, web, dirección...) y elimina las demás.
Buscar candidatas / Eliminar seleccionadas	Lista empresas sin ningún email válido (y, en modo seguro, sin teléfono) y permite borrarlas junto con sus dependencias.

Nota. Las acciones de borrado y fusión son **irreversibles**. Hacer copia de seguridad de la BD antes (`mysqldump`). Se recomienda ejecutar «Aplicar deduplicación» y «Fusionar» antes de cada campaña.

7. Pruebas realizadas

Las webs de origen no eran accesibles desde el entorno de pruebas, por lo que se reprodujo Empresite con un servidor HTTPS local que imita su HTML. El resto se probó contra MariaDB 10.11 real con el esquema de la sección 5.

Prueba	Resultado
Funciones auxiliares de ambos scrapers (URLs, paginación, filtros, nombres de fichero, extracción de fichas, teléfonos)	OK, salvo la paginación de Páginas Amarillas (sección 4).
Scraper Empresite de extremo a extremo con Chrome headless (2 páginas, 3 fichas, 1 repetida, 1 enlace legal)	OK: 3 empresas, repetida descartada, enlace legal excluido, checkpoints pg001/pg002 y acumulado correctos.
Consultor: carga, clasificación RE/posibles/enviados, teléfonos, estados	OK
Mantenimiento: análisis, deduplicación, eliminación de inválidos, redundancias, fusión, candidatas	OK (resultados verificados en la BD).
Envío (SMTP simulado) sin warm-up	OK: emails personalizados, estados actualizados a EN.
Plantilla con CSS entre llaves	Falla como se describe en 6.2.
Reactivación mensual EN → PE y cálculo de límites de warm-up	OK

8. Problemas conocidos y recomendaciones

Nivel	Problema	Recomendación
Alta	La carpeta selenium_profile_empresite/ (~360 MB, 2.075 ficheros) está en Git e incluye **Cookies, History y Login Data** del navegador.	Sacarla del repo (`git rm -r --cached`), añadirla a .gitignore y valorar limpiar el historial. Revisar si contiene sesiones o contraseñas guardadas.
Alta	Paginación de Páginas Amarillas rota (siempre la página 1).	Corregir la regex de construir_url (sección 4).
Media	Llaves en la plantilla HTML rompen el envío.	Usar estilos en línea o `{{ }}`; o sustituir format por reemplazos explícitos.
Media	Scraper Empresite lento (~27 s por página buscando banner de cookies).	Reducir timeout o aceptar cookies solo una vez.
Media	Con URLs `/localidad/…`, `tipo_empresa` queda como «localidad» y el email diría «vuestra actividad como localidad».	Rellenar la actividad al cargar en BD o derivarla de otra fuente.
Baja	Los emails no se normalizan al enviar (espacios, mayúsculas); el mismo email en dos filas puede tener estados distintos.	Ejecutar deduplicación antes de enviar; aplicar strip() al guardar.
Baja	En EXCLUDE_HTML_PATTERNS falta un «|» entre `use` y `contacto`.	Añadir el separador.
Baja	URL de webhook n8n escrita en WebScrapper_DAGM_ver3.py.	Moverla a .env o eliminar la versión antigua.
Baja	Varias versiones antiguas conviven en la raíz.	Mover a una carpeta legacy/ o confiar en el historial de Git.

8.1 Uso responsable

El scraping y el envío de comunicaciones comerciales deben respetar los términos de uso de los sitios de origen, el RGPD y la LSSI (art. 21): identificar al remitente, ofrecer un medio sencillo de baja y atender las solicitudes de oposición. Se recomienda añadir un enlace o instrucción de baja en la plantilla.